

AI Builders Bootcamp - Course Notes

These notes cover the core ideas behind modern language models: machine learning, deep learning, neural networks, transformers, attention, embeddings and retrieval augmented generation. They are the sample knowledge base for the practical RAG sessions.

Chapter 1: Machine Learning

Machine learning is a field of artificial intelligence in which a program improves its behaviour from data instead of from explicit rules written by a developer. A traditional program encodes the rules directly; a machine learning system is shown examples and derives the rules on its own by adjusting internal parameters.

There are three classic families of machine learning. Supervised learning trains on labelled pairs of input and expected output, and is used for classification and regression. Unsupervised learning receives unlabelled data and looks for structure, for example by clustering similar items together. Reinforcement learning trains an agent that takes actions in an environment and receives a reward signal, learning the policy that maximises long term reward.

Training a model means minimising a loss function, a number that measures how wrong the current predictions are. The optimiser computes the gradient of the loss with respect to every parameter and nudges the parameters in the direction that lowers the loss. This procedure is called gradient descent.

- Overfitting: the model memorises the training data and fails on new data.
- Underfitting: the model is too simple to capture the pattern in the data.
- Generalisation: performing well on data never seen during training.

Chapter 2: Deep Learning

Deep learning is the subset of machine learning that uses neural networks with many stacked layers. The word deep refers to the number of layers between the input and the output. Each layer transforms its input into a slightly more abstract representation, so early layers capture simple local patterns such as edges or character shapes while deeper layers capture high level concepts such as objects, topics or intent.

The key advantage of deep learning over classical machine learning is representation learning: the model discovers useful features automatically from raw data, so engineers no longer need to design features by hand. The price is a much larger appetite for data and compute, which is why deep learning became practical only once GPUs and large public datasets were widely available.

Chapter 3: Neural Networks

A neural network is a graph of simple computing units called neurons. A neuron multiplies each of its inputs by a weight, adds a bias, and passes the result through a non-linear activation function such as ReLU, sigmoid or GELU. Without the non-linearity a stack of layers would collapse into a single linear transformation and the network could not model complex relationships.

Learning happens through backpropagation. A forward pass computes the prediction and the loss. The backward pass then applies the chain rule of calculus layer by layer, from the output back to the input, to obtain the gradient of the loss with respect to every weight. The optimiser, usually a variant of stochastic gradient descent such as Adam, updates the weights using those gradients.

- Feedforward networks pass information in one direction only.
- Convolutional networks share weights across space and excel at images.
- Recurrent networks process sequences step by step and carry a hidden state.
- Transformers process a whole sequence in parallel using attention.

Chapter 4: Transformers

The transformer is the neural network architecture behind almost every modern large language model. It was introduced in 2017 in the paper Attention Is All You Need. Unlike recurrent networks, a transformer does not process tokens one after another; it reads the entire sequence at once and lets every token look at every other token. This makes training highly parallel on GPUs and removes the long dependency chains that made recurrent networks forget distant context.

A transformer block contains a multi head self attention sublayer followed by a small feedforward network, with residual connections and layer normalisation around each sublayer. Because attention is order independent by nature, positional information must be added explicitly, either as learned positional embeddings or with a scheme such as rotary position embeddings.

Three variants of the architecture are common. Encoder only models such as BERT produce rich representations of text and are used for classification, retrieval and embeddings. Decoder only models such as the GPT family predict the next token and are used for text generation. Encoder decoder models such as T5 map one sequence to another and suit translation or summarisation.

Chapter 5: The Attention Mechanism

Attention is the operation that lets a model decide, for every token, which other tokens matter. Each token is projected into three vectors: a query, a key and a value. The relevance of token j to token i is the dot product between the query of i and the key of j . Those scores are divided by the square root of the head dimension to keep them numerically stable, passed through a softmax so they sum to one, and then used as weights to average the value vectors. This is called scaled dot product attention.

Self attention means queries, keys and values all come from the same sequence, so the sequence attends to itself. Cross attention means the queries come from one sequence and the keys and values from another, which is how a decoder consults an encoder. Multi head attention runs several attention operations in parallel with different projections, so one head can track syntax while another tracks long range topic information, and the results are concatenated and projected back.

In a decoder, attention is causally masked: a token may only attend to itself and to tokens before it, otherwise the model could cheat by looking at the answer it is being asked to predict. The cost of attention grows with the square of the sequence length, which is the main reason context windows are limited and the main motivation for efficient attention variants such as FlashAttention.

Chapter 6: Embeddings

An embedding is a dense vector of floating point numbers that represents a piece of content in a way a machine can compare. A sentence embedding model maps a whole sentence or paragraph to a single vector, typically with 384, 768 or 1536 dimensions, so that texts with similar meaning land close together in the vector space even when they share no vocabulary.

Closeness is normally measured with cosine similarity, the cosine of the angle between two vectors. It ranges from minus one for opposite meaning to one for identical direction, and it ignores vector length, which makes it robust to differences in text length. When vectors are normalised to unit length, cosine similarity and the dot product are the same computation.

Embeddings are what make semantic search possible. A keyword index can only find documents that literally contain the words of the query, so a question about a car will not match a document about an automobile. An embedding index compares meaning, so the two match even with no shared token.

Chapter 7: Retrieval Augmented Generation

Retrieval augmented generation, usually shortened to RAG, is a pattern that gives a language model access to knowledge it was never trained on. Instead of relying only on the parameters learned during pretraining, the system searches an external knowledge base at question time and puts the retrieved passages into the prompt as context. The model then answers from that context.

RAG solves three concrete problems. It removes the knowledge cutoff, because the knowledge base can be updated at any time without retraining. It reduces hallucination, because the model is grounded in real passages and can be instructed to refuse when the context does not contain the answer. And it makes answers auditable, because every retrieved passage carries metadata such as the file name and the page number, which can be shown as a source citation.

A RAG system has two pipelines. The offline indexing pipeline runs once per document version: load the raw files, clean the text, split it into chunks, embed each chunk and store the vectors in a vector index. The online query pipeline runs on every question: embed the question, search the index for the nearest chunks, build a prompt from those chunks and the question, and send it to the language model for generation.

Chunking is the design decision with the largest effect on answer quality. Chunks that are too small lose the context needed to be understood on their own; chunks that are too large dilute the embedding, because a single vector has to represent several unrelated ideas at once, and they waste space in the context window. A small overlap between consecutive chunks keeps sentences that fall on a boundary from being cut in half.

RAG is not the only way to adapt a model. Prompt engineering changes only the instructions and is free but cannot add new facts. Fine tuning changes the weights and is good for teaching a style, a format or a skill, but it is expensive and stale facts stay baked in. RAG changes the knowledge available at inference time, which is the right tool when the facts are private, large or frequently updated.